

Guia Oficial de Engenharia Front-end — Santo Pegasus Soluciones

Localidade: São Paulo - SP

Versão: 2.0.0

Classificação: Uso Interno / Técnico

Data: 24 de Maio de 2024

1. Introdução: O papel do Front-end na Santo Pegasus

Na Santo Pegasus Soluciones, a engenharia de front-end transcende a simples construção de interfaces. Ela é o ponto de convergência entre a lógica de negócio complexa e a experiência do usuário final. Nosso objetivo é construir sistemas resilientes, escaláveis e de alta performance que reflitam a excelência técnica da nossa sede em São Paulo.

O engenheiro front-end da Santo Pegasus não é apenas um "desenvolvedor de telas", mas um arquiteto de software responsável pela integridade dos dados no cliente, pela acessibilidade e pela ponte de comunicação eficiente com nossos serviços de back-end.

1.1. O Front-end como Camada Estratégica de Negócio

Em um cenário competitivo onde a experiência digital é frequentemente o único ponto de contato entre a Santo Pegasus e seus clientes finais, o front-end assume um papel estratégico que vai muito além da entrega visual. Cada decisão técnica tomada nesta camada — seja a escolha de uma biblioteca de estado, a estratégia de cache de dados ou a forma como tratamos erros de rede — impacta diretamente métricas de negócio como taxa de conversão, retenção de usuários e percepção de qualidade da marca.

Nossos times de front-end trabalham em estreita colaboração com Product Managers, Designers de UX/UI e Engenheiros de Back-end, formando o que chamamos de "tríade de entrega". Essa colaboração multidisciplinar garante que decisões técnicas nunca sejam tomadas isoladamente, mas sempre considerando o impacto no usuário final e nos objetivos de negócio da empresa.

1.2. Desafios Técnicos Específicos

Os produtos da Santo Pegasus operam em contextos de alta criticidade — sistemas financeiros, plataformas de saúde e ferramentas administrativas de larga escala. Isso impõe desafios técnicos únicos:

- **Consistência de dados em tempo real:** Interfaces que refletem mudanças de estado quase instantaneamente, mesmo sob alta concorrência de usuários.
- **Resiliência a falhas de rede:** Aplicações que degradam graciosamente quando a conectividade é instável, sem perda de dados do usuário.

- **Escalabilidade de código:** Bases de código que crescem de forma sustentável, permitindo que múltiplos times trabalhem simultaneamente sem conflitos constantes.
- **Conformidade regulatória:** Aderência estrita a normas como LGPD, WCAG e, em alguns produtos, regulamentações específicas do setor financeiro (BACEN) e de saúde (ANVISA/CFM).

1.3. Filosofia de Engenharia

Acreditamos que código de qualidade é aquele que pode ser lido, entendido e modificado por qualquer engenheiro do time, não apenas por quem o escreveu originalmente. Por isso, priorizamos:

1. **Legibilidade sobre brevidade:** Código explícito é preferível a código "inteligente" e condensado.
2. **Testabilidade desde o design:** Componentes e hooks são desenhados pensando em como serão testados.
3. **Documentação viva:** Comentários e documentação devem explicar o "porquê", não o "o quê" — o código já expressa o "o quê".
4. **Feedback rápido:** Pipelines de CI/CD, linters e type-checking devem fornecer feedback em segundos, não em minutos.

2. Princípios de Engenharia

Adotamos princípios de design de software clássicos adaptados ao ecossistema React para garantir que o código seja sustentável a longo prazo.

2.1. SOLID em React

S — Single Responsibility Principle

Cada componente deve fazer apenas uma coisa. Se um componente gerencia estado global, faz chamadas de API e renderiza listas complexas, ele deve ser refatorado.

```
```tsx
```

```
// Violação: componente faz busca de dados, lógica de negócio e renderização
```

```
function UserProfile({ userId }: { userId: string }) {
 const [user, setUser] = useState<User | null>(null);
 const [loading, setLoading] = useState(true);
 useEffect(() => {
 fetch(`/api/users/${userId}`)
 .then((res) => res.json())
 .then((data) => {
 setUser(data);
 });
 });
}
```

```

 setLoading(false);
 });
}, [userId]);
if (loading) return Carregando...;
return <div>{user?.name}</div>;
}
// Correto: responsabilidades separadas
function useUser(userId: string) {
 return useQuery({
 queryKey: ["user", userId],
 queryFn: () => api.get<User>(`/users/${userId}`),
 });
}
function UserProfile({ userId }: { userId: string }) {
 const { data: user, isLoading } = useUser(userId);
 if (isLoading) return <Spinner />;
 return <UserProfileView user={user} />;
}
...

```

#### #### O — Open/Closed Principle

Componentes devem estar abertos para extensão (via props e composição), mas fechados para modificação interna.

```

````tsx
// Componente extensível via composição, sem alterar seu código-fonte
interface CardProps {
  children: React.ReactNode;
  className?: string;
}
function Card({ children, className }: CardProps) {
  return <div className={cn("rounded-lg border p-4", className)}>{children}</div>;
}
// Extensão via composição, sem modificar o componente Card
function UserCard({ user }: { user: User }) {
  return (
    <Card className="bg-slate-50">
      <CardHeader>{user.name}</CardHeader>
      <CardBody>{user.email}</CardBody>
    </Card>
  );
}

```

```

    </Card>
  );
}
...

```

L — Liskov Substitution Principle

Componentes que derivam de elementos base (ex: um `CustomButton` que estende `Button`) devem poder substituir seus pais sem quebrar a aplicação.

```

``tsx
type ButtonProps = React.ButtonHTMLAttributes<HTMLButtonElement>;
function Button({ className, ...props }: ButtonProps) {
  return <button className={cn("rounded-md px-4 py-2", className)} {...props} />;
}
// PrimaryButton deve ser substituível em qualquer lugar que use Button
function PrimaryButton(props: ButtonProps) {
  return <Button className="bg-pegasus-blue text-white" {...props} />;
}
...

```

I — Interface Segregation Principle

Não obrigue um componente a depender de props que ele não utiliza. Utilize TypeScript para definir contratos claros.

```

``tsx
// Interface "gorda" força consumo de props desnecessárias
interface TableProps {
  data: Row[];
  onEdit: (id: string) => void;
  onDelete: (id: string) => void;
  onExport: () => void; // Nem todo consumidor precisa de export
}
// Interfaces segregadas por responsabilidade
interface BaseTableProps {
  data: Row[];
}
interface EditableTableProps extends BaseTableProps {
  onEdit: (id: string) => void;
  onDelete: (id: string) => void;
}
interface ExportableTableProps extends BaseTableProps {

```

```

    onExport: () => void;
  }
  ...

```

D — Dependency Inversion Principle

Dependa de abstrações (hooks, contextos) em vez de implementações concretas dentro dos componentes de UI.

```

````tsx
// Componente depende diretamente da implementação do Axios
function OrderList() {
 const [orders, setOrders] = useState([]);
 useEffect(() => {
 axios.get("/api/orders").then((res) => setOrders(res.data));
 }, []);
 return {orders.map((o) => <li key={o.id}>{o.name})};
}
// Componente depende da abstração (hook), não da implementação
function useOrders() {
 return useQuery({ queryKey: ["orders"], queryFn: orderService.getAll });
}
function OrderList() {
 const { data: orders = [] } = useOrders();
 return {orders.map((o) => <li key={o.id}>{o.name})};
}
...

```

## 2.2. DRY, KISS e YAGNI

#### #### DRY (Don't Repeat Yourself)

*Abstraia lógicas repetitivas em Custom Hooks.*

```

````tsx
// Hook reutilizável para debounce, evitando duplicação em múltiplos componentes
function useDebounce<T>(value: T, delay = 300): T {
  const [debouncedValue, setDebouncedValue] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debouncedValue;
}

```

```

}
// Uso em qualquer componente de busca
function SearchInput() {
  const [term, setTerm] = useState("");
  const debouncedTerm = useDebounce(term);
  useEffect(() => {
    if (debouncedTerm) searchService.query(debouncedTerm);
  }, [debouncedTerm]);
  return <input value={term} onChange={(e) => setTerm(e.target.value)} />;
}
...

```

KISS (Keep It Simple, Stupid)

Evite "over-engineering". Se uma solução simples resolve o problema de forma legível, prefira-a.

```

````tsx
// Complexidade desnecessária para um caso simples
const isEven = (n: number) =>
 new Array(n).fill(null).reduce((acc, _, i) => (i === n - 1 ? n % 2 === 0 : acc), false);
// Solução simples e direta
const isEven = (n: number) => n % 2 === 0;
...

```

#### #### YAGNI (You Ain't Gonna Need It)

Não implemente funcionalidades ou abstrações "para o futuro" que não foram solicitadas no requisito atual.

```

````tsx
// Abstração prematura para "possíveis" múltiplos provedores de pagamento,
// quando o requisito atual só pede Pix
interface PaymentStrategy {
  process(): void;
}
class PixStrategy implements PaymentStrategy { process() {} }
class CreditCardStrategy implements PaymentStrategy { process() {} } // não solicitado ainda
class PaymentProcessor {
  constructor(private strategy: PaymentStrategy) {}
  execute() { this.strategy.process(); }
}
// Implementação direta para o requisito atual

```

```
function processPixPayment(orderId: string) {
  return api.post(`/orders/${orderId}/pix`);
}
...
---
```

3. Stack Tecnológico Padrão

Nossa stack é selecionada para oferecer o melhor equilíbrio entre produtividade do desenvolvedor e performance para o usuário.

- **React 18 (mínimo 18.2.0):** Utilizando as novas APIs de concorrência e Transitions.
- **TypeScript (mínimo 5.0):** Uso obrigatório de tipagem estrita (``strict: true``) para reduzir erros em tempo de execução.
- **Next.js (mínimo 14.x, App Router):** Framework principal para aplicações voltadas ao público (SEO e Performance via SSR/SSG).
- **Vite (mínimo 5.x):** Ferramenta de build para SPAs (Single Page Applications) internas e dashboards de baixa necessidade de SEO.
- **Node.js (mínimo LTS 20.x):** Runtime padrão para build e ferramentas de desenvolvimento.

3.1. Critérios de Decisão: Next.js vs Vite

A escolha entre os dois frameworks não é arbitrária. Utilizamos a seguinte matriz de decisão:

Critério	Next.js	Vite (SPA)
Necessidade de SEO / indexação	Obrigatório	Não aplicável
Renderização no servidor (SSR/SSG)	Sim, nativo	Não (apenas client-side)
Aplicação voltada ao público externo	Sim	Raramente
Dashboard interno / ferramenta administrativa	Possível, mas não ideal	Recomendado
Tempo de build e complexidade de infra	Maior (exige runtime Node)	Menor (build estático)
Roteamento baseado em arquivos	Sim (App Router)	Requer biblioteca (React Router)

Regra prática: se o produto será acessado por usuários anônimos via mecanismos de busca (marketing, landing pages, portal do cliente), utilize Next.js. Se o produto é uma ferramenta interna, autenticada, sem necessidade de indexação (ex: painel administrativo, dashboard de operações), utilize Vite.


```

useDebounce.ts
useMediaQuery.ts
services/          # Camada de comunicação com API
  api.ts           # Instância Axios
  userService.ts
  orderService.ts
stores/            # Zustand stores
  useAuthStore.ts
  useThemeStore.ts
types/             # Tipos globais e gerados via OpenAPI
  api-generated.d.ts
  domain.ts
lib/               # Utilitários (cn, formatters, etc.)
  utils.ts
styles/
  globals.css
tests/
  e2e/
    checkout.spec.ts  # Playwright
...

```

4.3. Separação de Lógica e UI

Adotamos o padrão de Container/Presenter via Hooks:

- A lógica de estado e chamadas de API residem em um Hook customizado (ex: `useUserDashboard.ts`).
- O componente React recebe os dados e funções via props, focando apenas na renderização.

Exemplo completo — `useUserDashboard.ts` (Container):

```

````tsx
// hooks/useUserDashboard.ts
import { useQuery } from "@tanstack/react-query";
import { useMemo, useState } from "react";
import { orderService } from "@services/orderService";
import { userService } from "@services/userService";
interface UseUserDashboardParams {
 userId: string;
}

```

```

export function useUserDashboard({ userId }: UseUserDashboardParams) {
 const [statusFilter, setStatusFilter] = useState<"all" | "pending" | "done">("all");
 const userQuery = useQuery({
 queryKey: ["user", userId],
 queryFn: () => userService.getByld(userId),
 });
 const ordersQuery = useQuery({
 queryKey: ["orders", userId, statusFilter],
 queryFn: () => orderService.getByUser(userId, statusFilter),
 enabled: !!userQuery.data,
 });
 const totalSpent = useMemo(
 () => ordersQuery.data?.reduce((sum, order) => sum + order.total, 0) ?? 0,
 [ordersQuery.data]
);
 return {
 user: userQuery.data,
 orders: ordersQuery.data ?? [],
 totalSpent,
 isLoading: userQuery.isLoading || ordersQuery.isLoading,
 isError: userQuery.isError || ordersQuery.isError,
 statusFilter,
 setStatusFilter,
 };
}
...

```

### Exemplo completo — `UserDashboard.tsx` (Presenter):

```

```tsx
// components/organisms/UserDashboard/UserDashboard.tsx
import { useUserDashboard } from "../useUserDashboard";
import { Spinner } from "@components/ui/spinner";
import { OrderList } from "@components/molecules/OrderList";
interface UserDashboardProps {
  userId: string;
}
export function UserDashboard({ userId }: UserDashboardProps) {
  const { user, orders, totalSpent, isLoading, isError, statusFilter, setStatusFilter } =

```

```

    useUserDashboard({ userId });
    if (isLoading) return <Spinner aria-label="Carregando painel do usuário" />;
    if (isError) return <p role="alert">Erro ao carregar dados do usuário.</p>;
    return (
      <section aria-labelledby="dashboard-title">
        <h1 id="dashboard-title">Olá, {user?.name}</h1>
        <p>Total gasto: R$ {totalSpent.toFixed(2)}</p>
        <OrderList orders={orders} statusFilter={statusFilter} onFilterChange={setStatusFilter} />
      </section>
    );
  }
  ...
  ---

```

5. Gerenciamento de Estado

Diferenciamos rigidamente o Estado do Servidor do Estado da UI.

- **Zustand:** Utilizado para estados globais da aplicação (ex: dados do usuário logado, preferências de tema, estado do carrinho). É leve e evita o boilerplate do Redux.
- **React Query (TanStack Query):** Utilizado para sincronização de dados com o back-end. Gerencia cache, estados de loading, error e re-fetch de forma automática.

5.1. Exemplo de Store Zustand

```

````tsx
// stores/useAuthStore.ts
import { create } from "zustand";
import { persist } from "zustand/middleware";
interface AuthState {
 user: { id: string; name: string; role: string } | null;
 isAuthenticated: boolean;
 setUser: (user: AuthState["user"]) => void;
 logout: () => void;
}
export const useAuthStore = create<AuthState>()(
 persist(
 (set) => ({
 user: null,
 isAuthenticated: false,
 setUser: (user) => set({ user, isAuthenticated: !!user }),

```

```

 logout: () => set({ user: null, isAuthenticated: false }),
 }},
 { name: "pegasus-auth-storage" }
)
);
...

```

## 5.2. Exemplo de Hook React Query

```

``tsx
// services/orderService.ts
import { api } from "../api";
import type { Order } from "@types/domain";
export const orderService = {
 getByUser: (userId: string, status: string) =>
 api.get<Order[]>(`/users/${userId}/orders`, { params: { status } }).then((r) => r.data),
};
// hooks/useOrders.ts
import { useQuery } from "@tanstack/react-query";
import { orderService } from "@services/orderService";
export function useOrders(userId: string, status: string) {
 return useQuery({
 queryKey: ["orders", userId, status],
 queryFn: () => orderService.getByUser(userId, status),
 staleTime: 60 * 1000, // 1 minuto de cache "fresco"
 retry: 2,
 });
}
...

```

## 6. Consumo de APIs e Segurança de Tokens

### 6.1. Axios e Interceptores

Utilizamos instâncias centralizadas do Axios. Interceptores são obrigatórios para:

- Injetar cabeçalhos de autenticação.
- Tratamento global de erros (ex: redirecionar para `/login` em caso de erro 401).
- Logging de performance em ambiente de staging.

**Exemplo completo de configuração:**

```

``tsx
// services/api.ts
import axios, { AxiosError } from "axios";
import { useAuthStore } from "@stores/useAuthStore";
export const api = axios.create({
 baseURL: process.env.NEXT_PUBLIC_API_URL,
 withCredentials: true, // envio de cookies HttpOnly
 timeout: 10_000,
});
// Interceptor de REQUEST
api.interceptors.request.use((config) => {
 const startTime = performance.now();
 config.headers["X-Request-Start"] = String(startTime);
 if (process.env.NODE_ENV === "staging") {
 console.info(`[API] → ${config.method?.toUpperCase()} ${config.url}`);
 }
 return config;
});
// Interceptor de RESPONSE
api.interceptors.response.use(
 (response) => {
 if (process.env.NODE_ENV === "staging") {
 const start = Number(response.config.headers["X-Request-Start"]);
 console.info(`[API] ← ${response.config.url} (${(performance.now() - start).toFixed(0)}ms)`);
 }
 return response;
 },
 (error: AxiosError) => {
 if (error.response?.status === 401) {
 useAuthStore.getState().logout();
 window.location.href = "/login";
 }
 if (error.response?.status === 403) {
 console.error("Acesso negado ao recurso solicitado.");
 }
 return Promise.reject(error);
 }
}

```

```
);
...
```

## 6.2. Autenticação JWT

Na Santo Pegasus, a segurança é prioridade.

- **Proibido:** Armazenar JWT no ``localStorage`` ou ``sessionStorage`` devido a ataques XSS.
- **Padrão:** O token deve ser trafegado via Cookies `HttpOnly`, configurados pelo back-end. O front-end apenas envia as credenciais com ``withCredentials: true``.

---

## 7. Tipagem de API com TypeScript

### 7.1. Interfaces e Tipos Utilitários

Toda comunicação com o back-end deve ser tipada, garantindo que erros de contrato sejam detectados em tempo de compilação.

```
```tsx  
// types/domain.ts  
export interface Order {  
  id: string;  
  userId: string;  
  status: "pending" | "processing" | "done" | "cancelled";  
  total: number;  
  createdAt: string;  
}  
  
// Tipos utilitários do TypeScript aplicados ao domínio  
export type CreateOrderPayload = Omit<Order, "id" | "createdAt">;  
export type OrderSummary = Pick<Order, "id" | "status" | "total">;  
export type PartialOrderUpdate = Partial<Pick<Order, "status">>;  
```
```

### 7.2. Geração Automática de Tipos a partir do OpenAPI/Swagger

Para evitar divergências manuais entre front-end e back-end, utilizamos geração automática de tipos a partir do contrato Swagger exposto pelos microsserviços.

```
```bash
```

Ferramenta padrão: openapi-typescript

```
npx openapi-typescript https://api.pegasus.com/swagger.json \  
  --output src/types/api-generated.d.ts  
```
```

```

````tsx
// Uso dos tipos gerados automaticamente
import type { components } from "@types/api-generated";
type ApiOrder = components["schemas"]["Order"];
const orderService = {
  getById: (id: string) => api.get<ApiOrder>(`/orders/${id}`).then((r) => r.data),
};
````

```

Esse comando é executado como parte do pipeline de CI, garantindo que qualquer mudança de contrato no back-end seja refletida automaticamente nos tipos do front-end, com falha de build caso haja quebra de compatibilidade.

---

## 8. Estilização e Design Tokens

### 8.1. Tailwind CSS + shadcn/ui

- **Tailwind CSS:** Para estilização rápida baseada em utilitários, garantindo consistência visual.
- **shadcn/ui:** Nossa biblioteca de componentes base. Os componentes são copiados para dentro do projeto (`components/ui`) para permitir total customização do código-fonte.

### 8.2. Design Tokens no `tailwind.config.js`

Valores como cores corporativas, espaçamentos e raios de borda devem ser definidos no arquivo `tailwind.config.js` para garantir que mudanças de marca sejam aplicadas globalmente de forma instantânea.

```

````js
// tailwind.config.js
/* @type {import('tailwindcss').Config} */
module.exports = {
  content: ["/src/**/*.ts,tsx"],
  theme: {
    extend: {
      colors: {
        "pegasus-blue": {
          50: "#eef4ff",
          500: "#2563eb",
          700: "#1d4ed8",
          900: "#1e3a8a",

```

```

    },
    "pegasus-gray": {
      100: "#f3f4f6",
      800: "#1f2937",
    },
  },
  spacing: {
    "18": "4.5rem",
    "22": "5.5rem",
  },
  borderRadius: {
    pegasus: "0.625rem",
  },
  fontFamily: {
    sans: ["Inter", "system-ui", "sans-serif"],
  },
},
},
},
plugins: [require("tailwindcss-animate")],
};
...

```

9. Gerenciamento de Formulários com React Hook Form + Zod

Formulários são pontos críticos de captura de dados e devem seguir um padrão único de validação e submissão.

9.1. Schema de Validação com Zod

```

````tsx
// schemas/orderFormSchema.ts
import { z } from "zod";
export const orderFormSchema = z.object({
 customerName: z.string().min(3, "Nome deve ter ao menos 3 caracteres"),
 email: z.string().email("E-mail inválido"),
 quantity: z.number().int().positive("Quantidade deve ser maior que zero"),
 paymentMethod: z.enum(["pix", "credit_card", "boleto"], {
 errorMap: () => ({ message: "Selecione um método de pagamento válido" }),
 }),
});

```



```
});
export type OrderFormData = z.infer<typeof orderFormSchema>;
...
```

## 9.2. Integração com React Hook Form

```
``tsx
// components/organisms/OrderForm/OrderForm.tsx
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { orderFormSchema, type OrderFormData } from "@schemas/orderFormSchema";
export function OrderForm({ onSubmit }: { onSubmit: (data: OrderFormData) => void }) {
 const {
 register,
 handleSubmit,
 formState: { errors, isSubmitting },
 } = useForm<OrderFormData>({
 resolver: zodResolver(orderFormSchema),
 });
 return (
 <form onSubmit={handleSubmit(onSubmit)} noValidate>
 <label htmlFor="customerName">Nome</label>
 <input id="customerName" {...register("customerName")}
 aria-invalid={!!errors.customerName} />

 {errors.customerName && {errors.customerName.message}}
 <label htmlFor="email">E-mail</label>
 <input id="email" type="email" {...register("email")} aria-invalid={!!errors.email} />
 {errors.email && {errors.email.message}}
 <button type="submit" disabled={isSubmitting}>
 {isSubmitting ? "Enviando..." : "Confirmar pedido"}
 </button>
 </form>
);
}

```

## 10. Acessibilidade (a11y)

Seguimos a norma WCAG 2.1 nível AA.

## 10.1. Checklist Detalhado WCAG 2.1 AA

- [ ] Perceptível: Contraste mínimo de 4.5:1 para texto normal e 3:1 para texto grande.
- [ ] Perceptível: Toda imagem informativa possui `alt` descritivo; imagens decorativas usam `alt=""`.
- [ ] Operável: Todos os elementos interativos são acessíveis via `Tab` e possuem foco visível (`:focus-visible`).
- [ ] Operável: Nenhuma armadilha de foco (focus trap) não intencional em modais.
- [ ] Compreensível: Mensagens de erro em formulários são anunciadas via `aria-live` ou `role="alert"`.
- [ ] Compreensível: Idioma da página declarado corretamente (`<html lang="pt-BR">`).
- [ ] Robusto: HTML validado sem elementos aninhados incorretamente.
- [ ] Uso correto de HTML semântico (`<main>`, `<nav>`, `<section>`).
- [ ] Atributos `aria-label` em elementos iconográficos.
- [ ] Suporte total à navegação via teclado (foco visível).
- [ ] Contraste de cores validado por ferramentas automatizadas (axe-core, Lighthouse).

## 10.2. Exemplos de Código com ARIA

```
``tsx
// Botão apenas com ícone precisa de aria-label descritivo
<button aria-label="Remover item do carrinho" onClick={handleRemove}>
 <TrashIcon aria-hidden="true" />
</button>

// Região de status dinâmico anunciada por leitores de tela
<div role="status" aria-live="polite">
 {isLoading ? "Carregando resultados..." : `${results.length} resultados encontrados`}
</div>

// Modal com foco gerenciado e rótulo acessível
<div role="dialog" aria-modal="true" aria-labelledby="modal-title">
 <h2 id="modal-title">Confirmar exclusão</h2>
 <button onClick={onClose} aria-label="Fechar modal">x</button>
</div>
...

```

## 11. Testes Automatizados

Nossa meta de cobertura de código é de 80%.

1. Vitest: Runner de testes unitários para funções puras e hooks.

**2. React Testing Library (RTL):** Para testes de integração de componentes, focando no comportamento do usuário e não na implementação interna.

**3. Playwright:** Para testes E2E (End-to-End) em fluxos críticos, como o checkout e o login.

### 11.1. Exemplo de Teste Unitário com Vitest

```
``tsx
// hooks/useDebounce.test.ts
import { describe, it, expect, vi } from "vitest";
import { renderHook, act } from "@testing-library/react";
import { useDebounce } from "../useDebounce";
describe("useDebounce", () => {
 it("deve atualizar o valor apenas após o delay", () => {
 vi.useFakeTimers();
 const { result, rerender } = renderHook(({ value }) => useDebounce(value, 300), {
 initialProps: { value: "a" },
 });
 rerender({ value: "ab" });
 expect(result.current).toBe("a"); // ainda não atualizou
 act(() => vi.advanceTimersByTime(300));
 expect(result.current).toBe("ab");
 vi.useRealTimers();
 });
});
...

```

### 11.2. Exemplo de Teste de Componente com Testing Library

```
``tsx
// components/organisms/OrderForm/OrderForm.test.tsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { OrderForm } from "../OrderForm";
describe("OrderForm", () => {
 it("exibe erro de validação quando e-mail é inválido", async () => {
 const onSubmit = vi.fn();
 render(<OrderForm onSubmit={onSubmit} />);
 await userEvent.type(screen.getByLabelText("Nome"), "Ana");
 await userEvent.type(screen.getByLabelText("E-mail"), "email-invalido");
 await userEvent.click(screen.getByRole("button", { name: /confirmar pedido/i }));
 });
});

```

```

 expect(await screen.findByRole("alert")).toHaveTextContent("E-mail inválido");
 expect(onSubmit).not.toHaveBeenCalled();
 });
});
...

```

### 11.3. Testes E2E com Playwright

Testes E2E validam fluxos críticos de ponta a ponta, simulando o comportamento real do usuário em um navegador.

```

````tsx
// tests/e2e/checkout.spec.ts
import { test, expect } from "@playwright/test";

test.describe("Fluxo de Checkout", () => {
  test("usuário completa uma compra com sucesso", async ({ page }) => {
    await page.goto("/produtos/123");
    await page.getByRole("button", { name: "Adicionar ao carrinho" }).click();
    await page.getByRole("link", { name: "Ver carrinho" }).click();
    await page.getByRole("button", { name: "Finalizar compra" }).click();
    await page.getByLabel("Nome completo").fill("Maria Silva");
    await page.getByLabel("E-mail").fill("maria@pegasus.com");
    await page.getByLabel("Método de pagamento").selectOption("pix");
    await page.getByRole("button", { name: "Confirmar pedido" }).click();
    await expect(page.getByText("Pedido confirmado com sucesso")).toBeVisible();
    await expect(page).toHaveURL(/\/pedidos\/\d+\/confirmacao/);
  });
});
...

```

12. GitFlow e Padrão de Commits

Seguimos o Conventional Commits para manter um histórico de mensagens legível e permitir a geração automática de changelogs.

Formato: `tipo(escopo): descrição curta`

- `feat`: Nova funcionalidade.
- `fix`: Correção de bug.
- `chore`: Manutenção de build, dependências, etc.
- `docs`: Mudanças na documentação.

13. Performance e Core Web Vitals

Aplicações da Santo Pegasus devem ser otimizadas para os indicadores do Google:

- **LCP (Largest Contentful Paint):** Otimização de imagens (Next/Image) e prioridade de carregamento.
- **FID (First Input Delay):** Minimização do tempo de execução do JavaScript no fio principal (Main Thread).
- **CLS (Cumulative Layout Shift):** Reserva de espaço para elementos carregados assincronamente para evitar pulos na interface.

13.1. Lazy Loading e Code Splitting

```
````tsx
// Carregamento assíncrono de componentes pesados
import { lazy, Suspense } from "react";
const ReportChart = lazy(() => import("@/components/organisms/ReportChart"));
function ReportsPage() {
 return (
 <Suspense fallback={<Spinner aria-label="Carregando gráfico" />}>
 <ReportChart />
 </Suspense>
);
}
...

````tsx
// Divisão de rotas em Next.js — code splitting automático por rota
// app/relatorios/page.tsx é servido como bundle independente do restante da aplicação
...
```

13.2. Otimização de Imagens com Next/Image

```
````tsx
import Image from "next/image";
function ProductBanner() {
 return (
 <Image
 src="/banners/promocao.jpg"
 alt="Promoção de produtos Pegasus"
 width={1200}
 height={400}
 />
);
}
```

```

 priority // usada para imagens acima da dobra (impacta LCP)
 placeholder="blur"
 blurDataURL="/banners/promocao-blur.jpg"
 />
);
}
...

```

## 14. Segurança e LGPD

### 14.1. Proteção no Navegador

- **XSS (Cross-Site Scripting):** Sanitização de qualquer entrada de usuário antes da renderização. Nunca utilize `dangerouslySetInnerHTML` sem validação rigorosa.
- **CSRF:** Proteção via tokens de sincronização e política de `SameSite` nos cookies.

### 14.2. Exemplo Prático de Sanitização

```

````tsx
import DOMPurify from "dompurify";
// Quando é estritamente necessário renderizar HTML dinâmico (ex: editor de texto rico)
function CommentContent({ rawHtml }: { rawHtml: string }) {
  const sanitizedHtml = DOMPurify.sanitize(rawHtml, {
    ALLOWED_TAGS: ["b", "i", "em", "strong", "p"],
    ALLOWED_ATTR: [],
  });
  return <div dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />;
}
// Preferência sempre por renderização segura via texto puro
function SafeComment({ text }: { text: string }) {
  return <p>{text}</p>; // React já escapa automaticamente
}
...
````
```

### 14.3. Conformidade com a LGPD

Como operamos em São Paulo, o front-end deve refletir a transparência exigida pela Lei Geral de Proteção de Dados:

- **Gestão de Consentimento:** Banners de cookies que permitem ao usuário optar por rastreamento analítico ou marketing.

- **Minimização de Dados:** Não coletar ou armazenar no estado do front-end dados sensíveis que não sejam estritamente necessários para a execução da tarefa atual.
- **Direito de Exclusão:** Interfaces claras para que o usuário solicite a exclusão de seus dados, integradas aos nossos endpoints de privacidade.

---

## 15. CI/CD para Front-end

### 15.1. Pipeline Padrão (GitHub Actions)

```yaml

.github/workflows/frontend-ci.yml

name: Frontend CI/CD

on:

pull_request:

branches: [main, develop]

push:

branches: [main]

jobs:

lint-test-build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with:

node-version: "20"

cache: "npm"

- run: npm ci

- run: npm run lint

- run: npm run type-check

- run: npm run test -- --coverage

- run: npm run build

e2e:

needs: lint-test-build

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- run: npx playwright install --with-deps

```

    - run: npm run test:e2e
docker-deploy:
  needs: [lint-test-build, e2e]
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t pegasus-frontend:${{ github.sha }} .
    - run: docker push registry.pegasus.com/frontend:${{ github.sha }}
...

```

15.2. Dockerfile de Referência

```
```dockerfile
```

### Dockerfile

```

FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm ci
COPY . .
RUN npm run build
FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next ./next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package.json ./
RUN npm ci --omit=dev
EXPOSE 3000
CMD ["npm", "start"]
...

```

---

## 16. Monitoramento de Erros em Produção

**Utilizamos Sentry como ferramenta padrão de rastreamento de erros e performance no front-end.**

```

```tsx
// lib/sentry.ts
import { Sentry } from "@sentry/nextjs";

```



```

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  tracesSampleRate: 0.2,
  environment: process.env.NODE_ENV,
  beforeSend(event) {
    // Nunca enviar dados sensíveis (LGPD) para o Sentry
    delete event.user?.email;
    return event;
  },
});
...

````tsx
// Captura manual de erros em pontos críticos
try {
 await orderService.create(payload);
} catch (error) {
 Sentry.captureException(error, { tags: { feature: "checkout" } });
 throw error;
}
...

```

Alertas são configurados para notificar o time responsável via Slack sempre que a taxa de erros de uma release exceder o limiar de 1% das sessões ativas.

---

## 17. Glossário de Termos Técnicos

- **SSR (Server-Side Rendering):** Renderização do HTML no servidor a cada requisição.
- **SSG (Static Site Generation):** Geração do HTML em tempo de build, servido como arquivo estático.
- **Hydration:** Processo de "ativação" do JavaScript React sobre o HTML já renderizado.
- **Code Splitting:** Divisão do bundle JavaScript em partes menores, carregadas sob demanda.
- **Custom Hook:** Função JavaScript que utiliza hooks do React para encapsular lógica reutilizável.
- **Container/Presenter:** Padrão que separa lógica de estado (Container) da renderização visual (Presenter).
- **Stale Time:** Tempo em que um dado em cache é considerado "fresco" pelo React Query.

- **Bounded Context:** Fronteira lógica de um domínio de negócio (termo herdado do Domain-Driven Design).
- **XSS (Cross-Site Scripting):** Vulnerabilidade que permite injeção de scripts maliciosos em páginas.
- **CSRF (Cross-Site Request Forgery):** Ataque que induz o usuário a executar ações não intencionais em um site autenticado.
- **WCAG: Web Content Accessibility Guidelines** — diretrizes internacionais de acessibilidade web.
- **LCP, FID, CLS:** Métricas do Core Web Vitals do Google relacionadas a performance percebida.
- **Zod:** Biblioteca de validação e inferência de esquemas em TypeScript.
- **OpenAPI/Swagger:** Especificação padrão para documentação e contrato de APIs REST.

---

**Aprovado por:**

*Departamento de Engenharia de Software — Santo Pegasus Soluciones*

*Diretoria de Tecnologia (CTO)*